



Projet Biathlon

CAHIER DES TESTS

PROPOSÉE À : Thierry LEGOFF



- 3 AVRIL 2026 -

Proposée par :

Marin Weis
Nolann Lescop
Matthieu Gouélo
Lucien Carré

TABLE DES MATIÈRES

1. Présentation de l'équipe
2. Objectif du cahier des tests
3. Environnement des tests
4. Tests unitaires
5. Tests fonctionnels
6. Test d'intégration
7. Tests de sécurité

NOTRE ÉQUIPE



- Matthieu GOUÉLO -

Scrum Master (chef de projet) & Développeur FullStack
Contact : gouelo.e2400357@etud.univ-ubs.fr



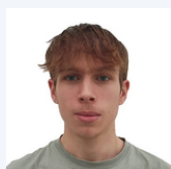
- Marin WEIS -

Responsable Communication & Développeur FullStack
Contact : weis.e2400483@etud.univ-ubs.fr



- Nolann LESCOP -

Développeur FullStack
Contact : lescop.e2403222@etud.univ-ubs.fr



- Lucien CARRE -

Développeur FullStack
Contact : carre.e2402145@etud.univ-ubs.fr



OBJECTIF

Ce document présente les tests réalisés dans le cadre du projet SAE Biathlon afin de vérifier le bon fonctionnement, la fiabilité et la sécurité de l'application avant sa livraison au client.

Dans un premier temps, sera présenté l'environnement dans lequel les tests ont été réalisés.

Ensuite les tests unitaires mettant à l'épreuve des parties du code isolées. Les tests fonctionnels réalisés dans le cadre d'un scénario prédéfini. Tests d'intégration qui permettent de vérifier que les différentes composantes de l'application fonctionnent et communiquent entre elles. Enfin les tests de sécurité pour prévenir toute forme de bug ou de comportement inattendu de l'application.

ENVIRONNEMENT DE TEST

Les tests ont été réalisés dans les environnements suivant :

Depuis Android Studio

- Emulateur Android Pixel Tablet
 - Version Android 16.0 "Baklava"
 - Version de l'application : 1.0
 - Période des tests : Sprint 1 à 5
 - Outils : JUnit, Robolectric.
-



Depuis la tablette du collègue

- Tablette Samsung Galaxy Tab A
- Version Android 11.0 "Red Velvet Cake"
- Version de l'application : 1.0
- Période des tests : Sprint 1 à 5



TESTS UNITAIRES

Les tests unitaires ont pour objectif de tester des fonctions / méthodes / classes de façon isolées

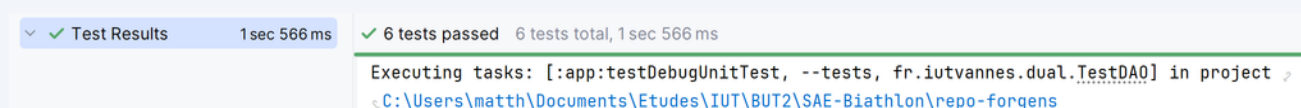
I - Test D.A.O

Dans ces tests, on vérifie la couche persistance (Room + DAO) indépendamment de l'interface.

Ces tests permettent de vérifier les opérations CRUD (Create, Read, Update, Delete) sur la base de données Room, en utilisant une base de données en mémoire afin de ne pas impacter les données réelles de l'application.

Les tests sont exécutés à l'aide de JUnit et Robolectric, et utilisent des coroutines pour simuler le comportement réel de l'application.

Résultat d'exécution :



```
✓ Test Results 1 sec 566 ms ✓ 6 tests passed 6 tests total, 1 sec 566 ms
Executing tasks: [:app:testDebugUnitTest, --tests, fr.iutvannes.dual.TestDAO] in project
C:\Users\matth\Documents\Etudes\IUT\BUT2\SAE-Biathlon\repo-forgens
```

TEST UNITAIRES - TEST DAO

Méthode testées	Description	Résultat attendu	Finalité
Insert / getEleveById	Insertion d'un élève puis récupération par ID	L'élève est récupéré avec les bonnes informations	OK
update	Modification du nom d'un élève existant	Les données sont mises à jour	OK
delete	Suppression d'un élève	L'élève n'existe plus en base	OK
insert / getProfByEmail	Insertion d'un professeur puis recherche par email	Le professeur est correctement récupéré	OK
insert / getSeanceById	Création d'une séance liée à un professeur	La séance est correctement enregistrée	OK
insert / getResultatById	Enregistrement d'un résultat pour un élève et une séance	Le résultat est correctement enregistré	OK

TESTS UNITAIRES - TEST VMA

II - Test VMA

Des tests unitaires ont été réalisés afin de valider le bon fonctionnement de l'algorithme de calcul de la VMA des élèves.

L'objectif est de vérifier que la méthode *calculer(distance, temps)* retourne une valeur correcte en km/h dans des situations nominales, et qu'elle réagit correctement aux entrées invalides.

Les tests couvrent deux catégories :

- Cas normaux : distances et temps cohérents permettant de vérifier le calcul attendu.
- Cas d'erreur : gestion des entrées impossibles (temps nul ou négatif) et vérification du comportement en cas d'objet non initialisé (*null*).

Le résultat est comparé à une valeur attendue avec une tolérance (écart $< 0,001$) afin de tenir compte des arrondis et des calculs en nombres réels.

TEST UNITAIRES - TEST VMA

Élément testé	Données en entrée	Résultat attendu	Finalité
<i>VMA.calculer(distance, temps)</i>	distance = 1000, temps = 300	VMA = 12,0 km/h	OK
<i>VMA.calculer(distance, temps)</i>	distance = 500, temps = 100	VMA = 18,0 km/h	OK
<i>VMA.calculer(distance, temps)</i>	distance = 1000, temps = 0	Exception attendue	OK
<i>VMA.calculer(distance, temps)</i>	distance = 1000, temps = -10	Exception attendue	OK
<i>Appel méthode sur objet VMA nul</i>	vma = null	Exception attendue (NullPointerException)	OK

TEST UNITAIRES - TEST VMA

Résultat d'exécution :

```
Test calculer(distance, temps)
Cas normaux
Test réussi : 12.0 km/h
Test réussi : 18.0 km/h
Cas d'erreur
Test réussi (exception capturée : IllegalArgumentException)
Test réussi (exception capturée : IllegalArgumentException)
Test réussi (exception capturée : NullPointerException)
BUILD SUCCESSFUL in 2s
29 actionable tasks: 1 executed, 28 up-to-date
```

TESTS UNITAIRES - CSV READER

III - Test CSV Reader

Des tests unitaires ont été réalisés afin de valider le bon fonctionnement du module d'importation des élèves à partir de fichiers CSV.

Les tests portent sur la méthode *read(InputStream)* de la classe *CsvStudentReader*, dont le rôle est de lire un flux CSV et de produire une liste d'objets *StudentDraft*.

Différents scénarios ont été testés afin de garantir la robustesse du traitement :

- Cas normaux : lecture de fichiers CSV correctement structurés contenant plusieurs élèves.
- Variante de format : présence de colonnes supplémentaires non utilisées et de lignes vides dans le fichier.
- Cas d'erreur : absence de colonnes obligatoires entraînant une exception.

Les fichiers CSV sont simulés en mémoire à l'aide de flux (*ByteArrayInputStream*), ce qui permet de tester la logique de lecture sans dépendre de fichiers physiques.

TEST UNITAIRES - CSV READER

Élément testé	Données en entrée	Résultat attendu	Finalité
<i>CsvStudentReader.read()</i>	CSV avec colonnes Prénom;Nom;Classe	2 élèves correctement lus	OK
<i>CsvStudentReader.read()</i>	CSV avec colonne DateNaissance en plus	2 élèves lus, colonne ignorée	OK
<i>CsvStudentReader.read()</i>	CSV contenant une ligne vide	2 élèves lus, ligne ignorée	OK
<i>CsvStudentReader.read()</i>	CSV sans colonne Prénom	Exception levée	OK

TEST UNITAIRES - CSV READER

Résultat d'exécution :

```
===== Test CsvStudentReader.read() =====  
Cas normal : 2 élèves valides  
Test réussi : 2 élèves lus  
  -> Marin Weis (4A)  
  -> Matthieu Gouélo (3B)  
Cas normal : 2 élèves valides + une colonne à sauter  
Test réussi : 2 élèves lus  
  -> Marin Weis (4A)  
  -> Matthieu Gouélo (3B)  
Cas avec ligne vide  
Test réussi : 2 élèves lus  
  -> Léa Dupont (4A)  
  -> Sam Bernard (5C)  
Cas d'erreur : colonne 'Prénom' manquante  
Test réussi (exception capturée : IllegalArgumentException)  
BUILD SUCCESSFUL in 2s  
29 actionable tasks: 1 executed, 28 up-to-date
```

TESTS UNITAIRES - EXCEL READER

IV - Test Excel Reader

Tests réalisés afin de valider le bon fonctionnement du module d'importation des élèves à partir de fichiers Excel (.xlsx).

Les tests portent sur la méthode *read(InputStream)* de la classe *XlsStudentReader*, dont le rôle est de lire un fichier Excel et de produire une liste d'objets *StudentDraft*.

Afin de garantir des tests reproductibles et indépendants de fichiers réels, les classeurs Excel sont générés directement en mémoire à l'aide de la bibliothèque Apache POI (*XSSFWorkbook*), puis convertis en *InputStream*.

Plusieurs scénarios ont été testés pour vérifier la robustesse de la lecture :

- Cas normal : lecture d'un fichier contenant des colonnes attendues et plusieurs élèves valides.
- Cas limite : présence d'une ligne vide, qui doit être ignorée.
- Cas d'erreur : absence d'une colonne obligatoire (ex : "Prénom"), qui doit déclencher une exception.

TEST UNITAIRES - EXCEL READER

Élément testé	Données en entrée	Résultat attendu	Finalité
<i>XlsStudentReader.read()</i>	Classeur avec colonnes Prénom; Nom; Classe et 2 lignes élèves	2 élèves correctement lus	OK
<i>XlsStudentReader.read()</i>	Classeur avec 3 lignes dont 1 ligne vide (valeurs vides)	2 élèves lus, la ligne vide ignorée	OK
<i>XlsStudentReader.read()</i>	Classeur sans colonne Prénom (ex : seulement Nom; Classe)	Exception levée	OK

TEST UNITAIRES - EXCEL READER

Résultat d'exécution :

```
===== Test ExcelStudentReader.read() =====  
Cas normal : 2 élèves valides  
  
Test réussi : 2 élèves lus  
  -> Glen Potay (4A)  
  -> Nolann Les-Cops (3B)  
Cas avec ligne vide  
Test réussi : 2 élèves lus  
  -> Glen Potay (4A)  
  -> Nolann Les-Cops (5C)  
Cas d'erreur : colonne 'Prénom' manquante  
Test réussi (exception capturée : IllegalArgumentException)  
BUILD SUCCESSFUL in 3s  
29 actionable tasks: 1 executed, 28 up-to-date
```


TESTS UNITAIRES - ODF READER

V - Test ODF Reader

Des tests unitaires ont été réalisés afin de valider le bon fonctionnement du module d'importation des élèves à partir de fichiers LibreOffice / OpenDocument *Spreadsheet* (.ods).

Les tests portent sur la méthode *read(InputStream)* de la classe *OdfStudentReader*, dont le rôle est de lire un tableur ODS et de produire une liste d'objets *StudentDraft*.

Afin de rendre les tests indépendants de fichiers externes, les tableurs ODS sont générés directement en mémoire à l'aide de la librairie ODF Toolkit (*SpreadsheetDocument*).

Plusieurs scénarios ont été définis :

- Cas normal : fichier correctement structuré contenant plusieurs élèves valides.
- Cas limite : présence d'une ligne vide, qui doit être ignorée lors de la lecture.
- Cas d'erreur : absence d'une colonne obligatoire (ex : "Prénom"), ce qui doit déclencher une exception.

TEST UNITAIRES - ODF READER

Élément testé	Données en entrée	Résultat attendu	Finalité
<i>OdfStudentReader.read()</i>	ODS avec colonnes Prénom; Nom; Classe et 2 lignes élèves	2 élèves correctement lus	OK
<i>OdfStudentReader.read()</i>	ODS avec une ligne vide entre 2 élèves	2 élèves lus, la ligne vide ignorée	OK
<i>OdfStudentReader.read()</i>	ODS sans colonne Prénom (ex : Nom; Classe)	Exception levée	OK

TEST UNITAIRES - ODF READER

Résultat d'exécution :

```
===== Test OdfStudentReader.read() =====  
Cas normal : 2 élèves valides  
Test réussi : 2 élèves lus  
  -> Nolann LesCops (4A)  
  -> Matth Gouélo (3B)  
Cas avec ligne vide  
Test réussi : 2 élèves lus  
  -> Nolann LesCops (4A)  
  -> Matthieu Gouélo (5C)  
Cas d'erreur : colonne 'Prénom' manquante  
Test réussi (exception capturée : IllegalArgumentException)  
BUILD SUCCESSFUL in 3s  
29 actionable tasks: 1 executed, 28 up-to-date
```

TESTS UNITAIRES - DETERMINER PARCOURS

V - Test Déterminer Parcours

Des tests unitaires ont été réalisés afin de valider le bon fonctionnement de l'algorithme déterminant le parcours attribué à l'élève en fonction de sa VMA.

Les tests portent sur la méthode *determinerParcours(VMA)* du fragment *EleveProfilFragment*, dont le rôle est d'attribuer un parcours dont la distance est adaptée à la VMA de l'élève.

Plusieurs scénarios ont été définis :

- Cas normal : la VMA de l'élève en paramètre sont des VMA en milieu d'intervall
- Cas limite : la VMA se trouve exactement à un point de pivot entre 2 parcours
- Cas d'erreur : la VMA est négative.

TEST UNITAIRES - DETERMINER PARCOURS

Élément testé	Données en entrée	Résultat attendu	Finalité
<i>Parcours.determine rParcours()</i>	VMA au milieu d'un intervalle défini (ex : 10.0f, 12.0f, 14.8f)	Le parcours correspondant au bon palier	OK
Parcours.determin erParcours()	VMA exacte sur une valeur de changement de palier (Pivot) (ex : 10.5f, 11.5f)	Bascule sur le parcours du palier supérieur (ex : 10.5f renvoie "Plots verts")	OK
<i>Parcours.determine rParcours()</i>	VMA sur la limite maximale stricte (ex : 15.0f)	"Plots rouges"	OK
<i>Parcours.determine rParcours()</i>	VMA juste au- dessus de la limite maximale (ex : 15.1f)	"Grand tour"	OK
<i>Parcours.determine rParcours()</i>	VMA négative ou aberrante (ex : -2.0f)	"Coupelles jaunes" (L'application ne crash pas et ne lève aucune exception)	OK

TEST UNITAIRES - DETERMINER PARCOURS

Résultat d'exécution :

```
Test determinerParcours(vma)
--- Cas normaux (Milieu d'intervalle) ---
Test réussi pour VMA 10.0 : Coupelles jaunes
Test réussi pour VMA 11.0 : Plots verts
Test réussi pour VMA 12.0 : Coupelles bleues
Test réussi pour VMA 13.0 : Plots bleus
Test réussi pour VMA 14.0 : Coupelles rouges
Test réussi pour VMA 14.8 : Plots rouges
Test réussi pour VMA 16.0 : Grand tour
--- Cas limites (Pivots exacts) ---
Test réussi pour VMA 10.5 : Plots verts
Test réussi pour VMA 11.5 : Coupelles bleues
Test réussi pour VMA 12.5 : Plots bleus
Test réussi pour VMA 13.5 : Coupelles rouges
Test réussi pour VMA 14.5 : Plots rouges
Test réussi pour VMA 15.0 : Plots rouges
Test réussi pour VMA 15.1 : Grand tour
--- Cas d'erreur ---
Test réussi pour VMA -2.0 : Coupelles jaunes
```

TESTS UNITAIRES - NORMALISER TEXTE

V - Test Déterminer Parcours

Des tests unitaires ont été réalisés afin de valider le bon fonctionnement de l'algorithme de normalisation de texte en javascript.

Les tests portent sur la méthode *normaliserTexte(texte)* du fichier main.js, dont le rôle est de nettoyer les entrées dans les champs de textes des élèves afin d'y retirer les apostrophes, accents, espaces en trop, trait d'unions et majuscules. Cela permet de rendre la recherche d'élèves plus souples et de ne pas prendre en compte ces éléments syntaxique qui sont souvent omis lors de la saisie clavier

Plusieurs scénarios ont été définis :

- Cas avec accent : l'entrée contient des accents
- Cas avec des caractère spéciaux : l'entrée contient des traits d'unions / apostrophes
- Cas avec des espaces en trop : l'entrée contient plus d'espace qu'attendu
- Cas combinés : l'entrée contient des caractères spéciaux + espaces
- Cas d'erreur : valeur null, indéfinie

TEST UNITAIRES - NORMALISER TEXTE

Élément testé	Données en entrée	Résultat attendu	Finalité
<i>normaliserTexte()</i>	Chaîne avec majuscules et accents divers (ex : "Éléonore", "AdÈIE", "François")	Chaîne entièrement en minuscules et sans aucun accent (ex : "eleonore", "adele", "francois")	OK
<i>normaliserTexte()</i>	Chaîne contenant des tirets ou des apostrophes (ex : "Jean-Baptiste", "D'Artagnan")	Les tirets et apostrophes sont remplacés par des espaces simples (ex : "jean baptiste", "d artagnan")	OK
<i>normaliserTexte()</i>	Chaîne avec des espaces superflus au début et à la fin (ex : " Espaces en trop ")	Les espaces aux extrémités sont supprimés (ex : "espaces en trop")	OK
<i>normaliserTexte()</i>	Chaîne complexe combinant tous les cas précédents (ex : " L'Étoile-Filante ")	Chaîne totalement formatée et nettoyée (ex : "l etoile filante")	OK
<i>normaliserTexte()</i>	Valeur non textuelle, nulle ou non définie (ex : null, undefined)	Une exception est levée (le test la capture avec succès)	OK

TEST UNITAIRES - NORMALISER TEXTE

Résultat d'exécution :

```
=== Test normaliserTexte(texte) ===  
  
--- Cas normaux ---  
Test réussi pour "Éléonore" : "eleonore"  
Test réussi pour "AdÈlE" : "adele"  
Test réussi pour "François" : "francois"  
  
--- Cas avec caractères spéciaux ---  
Test réussi pour "Jean-Baptiste" : "jean baptiste"  
Test réussi pour "D'Artagnan" : "d artagnan"  
  
--- Cas des espaces ---  
Test réussi pour "  Espaces en trop  " : "espaces en trop"  
  
--- Cas combiné ---  
Test réussi pour "  L'Étoile-Filante  " : "l etoile filante"  
  
--- Cas d'erreur ---  
Test réussi pour "null" (exception capturée : TypeError)  
Test réussi pour "undefined" (exception capturée : TypeError)
```

TESTS FONCTIONNELS

Les tests fonctionnels ont pour objectif de vérifier que les fonctionnalités demandées par le client fonctionne

I - Test Inscription / Connexion

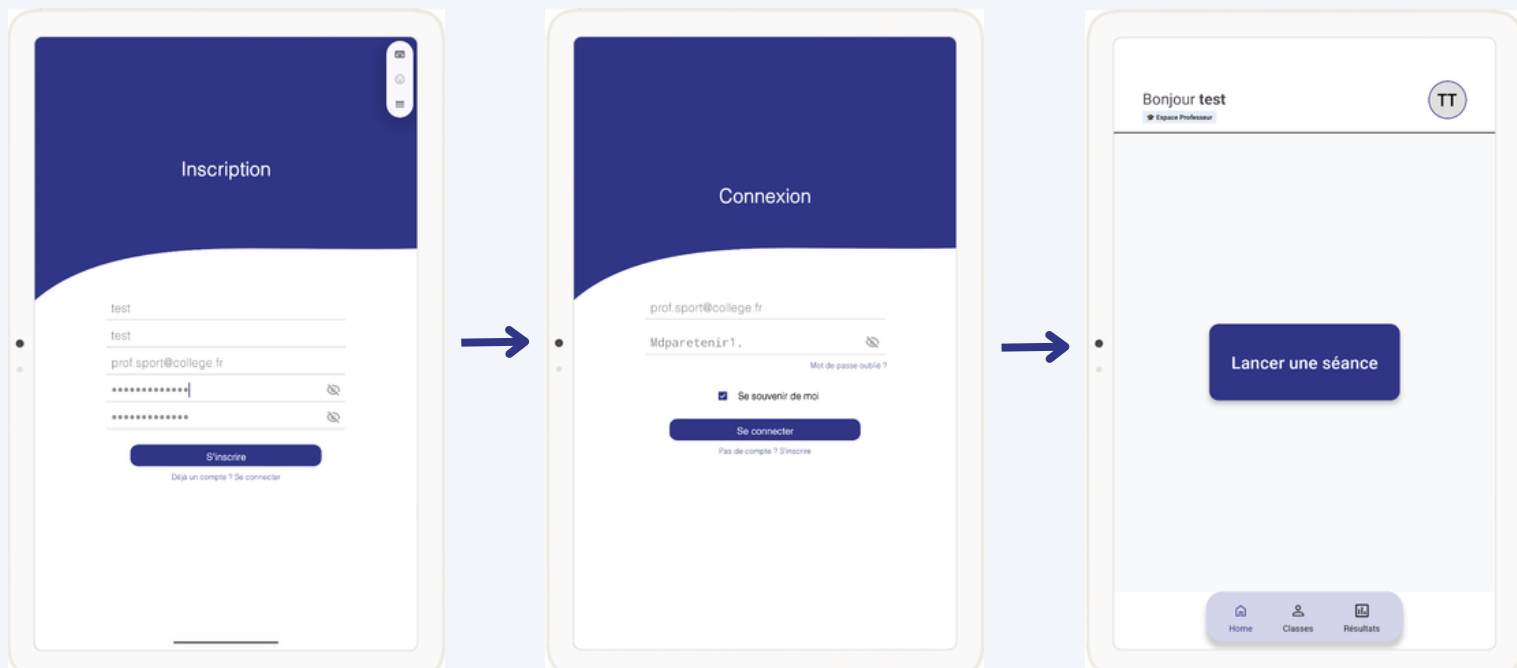
Précondition : application installée (voir manuel d'installation)

Données utilisées :

Email : prof.sport@college.fr (email fictif)

mdp : Mdparetenir1.

Résultat attendu : compte créé, redirection vers écran de connexion. Après la connexion redirection vers l'écran d'accueil.



TESTS FONCTIONNELS

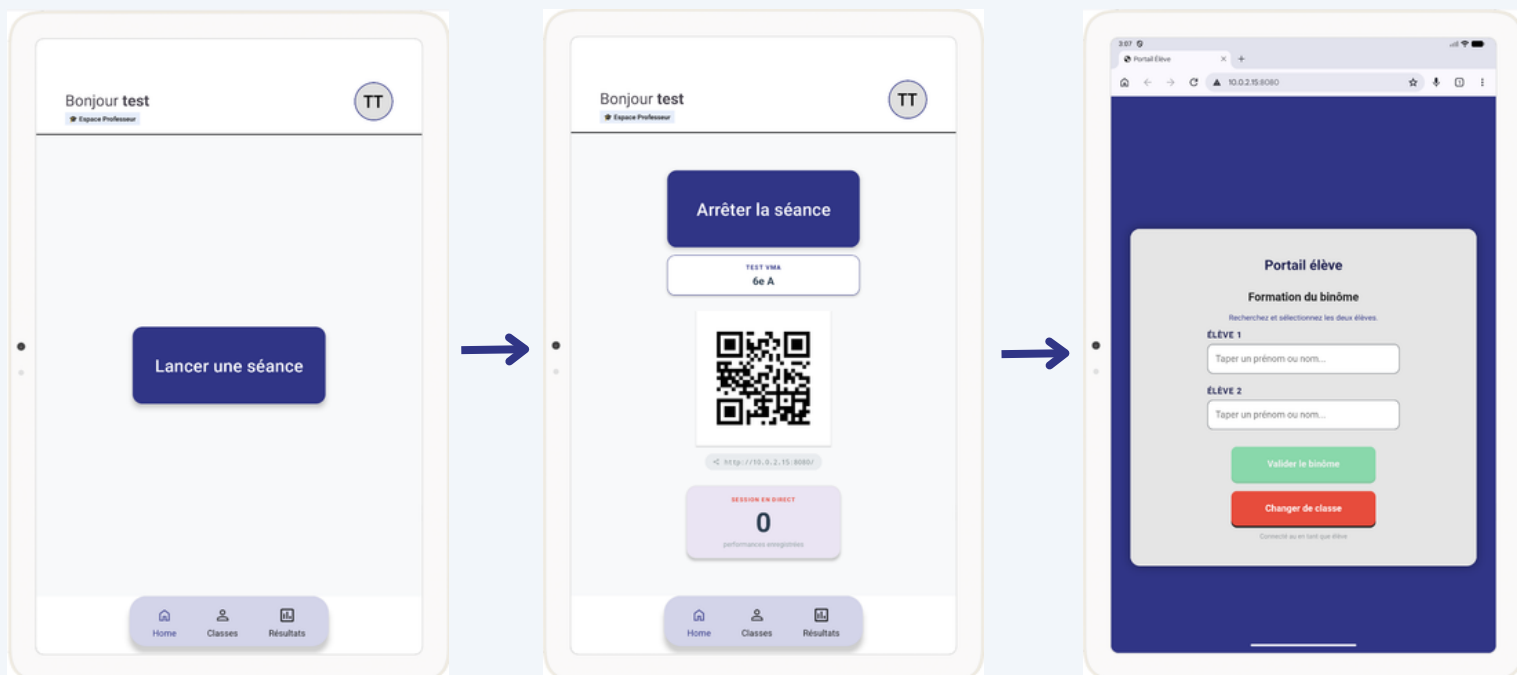
II - Test Lancement de séance

Précondition : application installée (voir manuel d'installation)

Dans ce test, on lance une séance via le bouton sur l'interface d'accueil.

Résultat attendu : l'url pour accéder au portail élève est généré ainsi que le QR code associé.

Le lien une fois entré dans le navigateur web, ouvre l'interface élève.



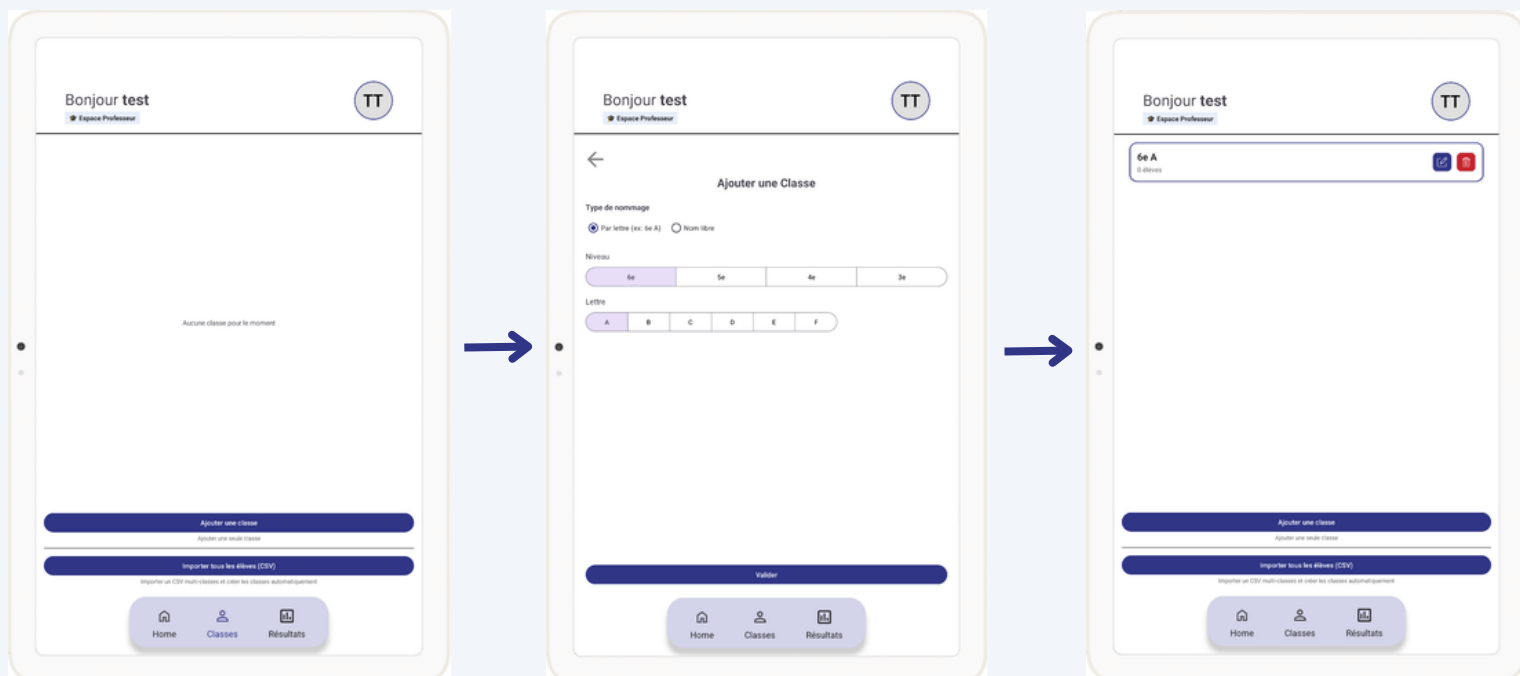
TESTS FONCTIONNELS

III - Test Ajout d'une classe

Précondition : application installée (voir manuel d'installation)

Dans ce test, on ajoute une classe

Résultat attendu : après création, la classe apparaît dans la liste de classe



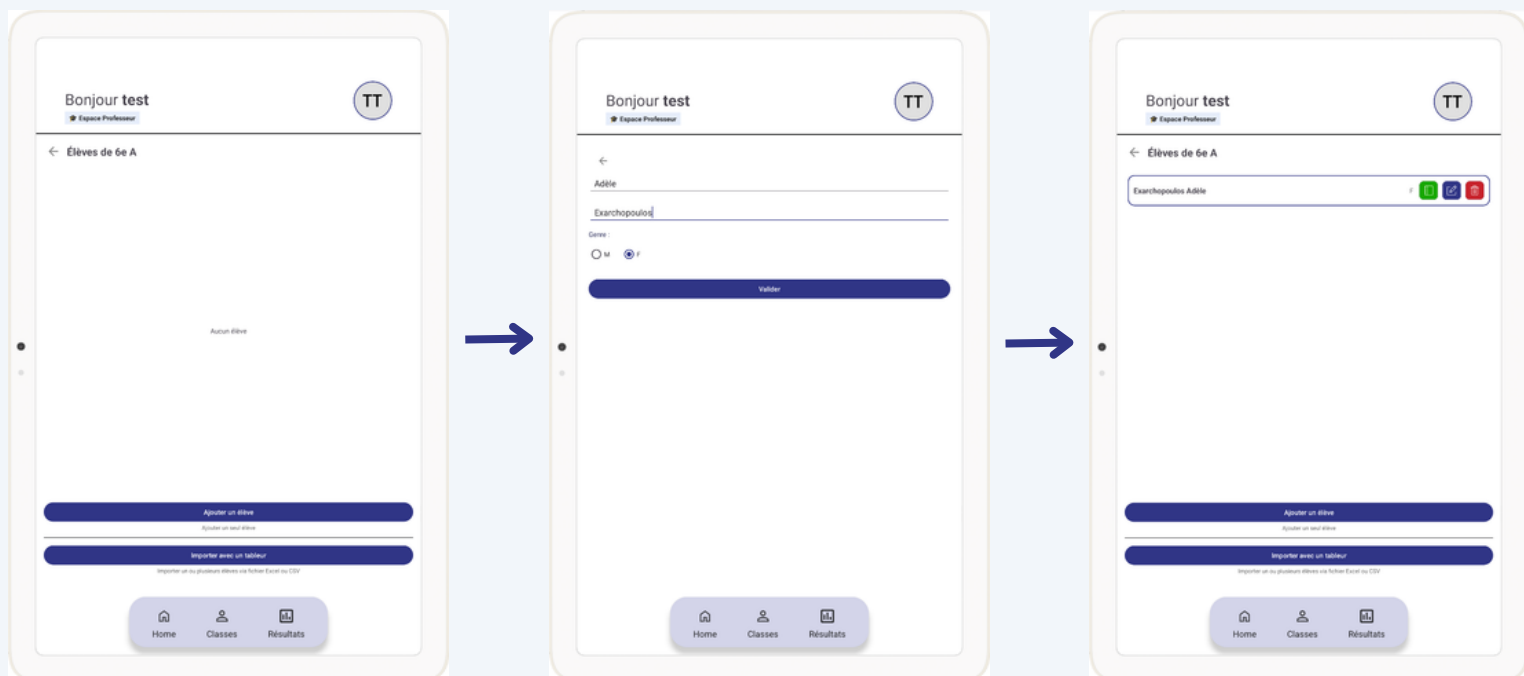
TESTS FONCTIONNELS

III - Test Ajout d'élèves

Précondition : application installée (voir manuel d'installation)

Dans ce test, on ajoute des élèves manuellement.

Résultat attendu : après création l'élève apparaît dans la liste des élèves de la classe.



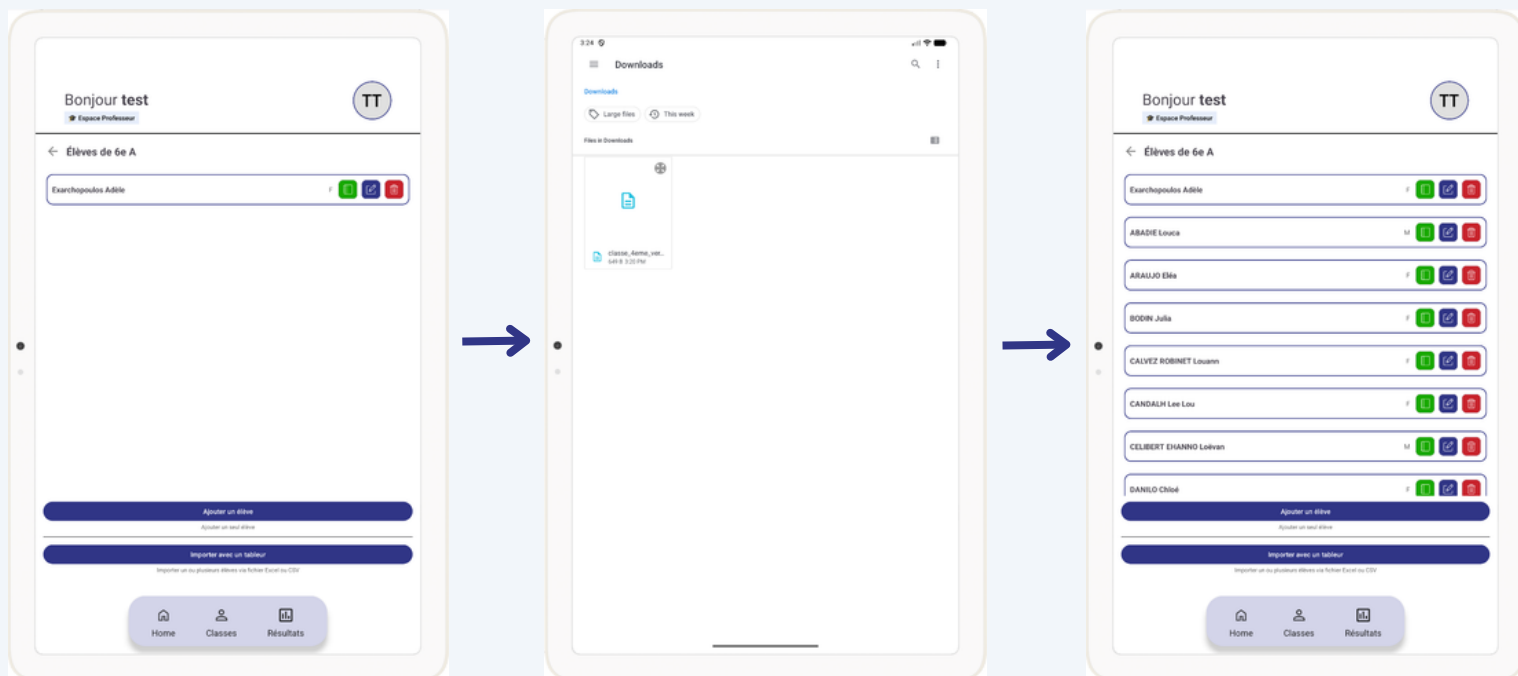
TESTS FONCTIONNELS

III - Test Ajout d'élèves

Précondition : application installée (voir manuel d'installation)

Dans ce test, on ajoute des élèves manuellement.

Résultat attendu : après import, les élèves apparaissent dans la liste de la classe.



TESTS D'INTÉGRATION

Les tests d'intégration ont pour objectif de vérifier le bon fonctionnement global de l'application Dual, en s'assurant que les différentes composantes du système communiquent correctement entre elles.

Ces tests portent notamment sur l'interaction entre :

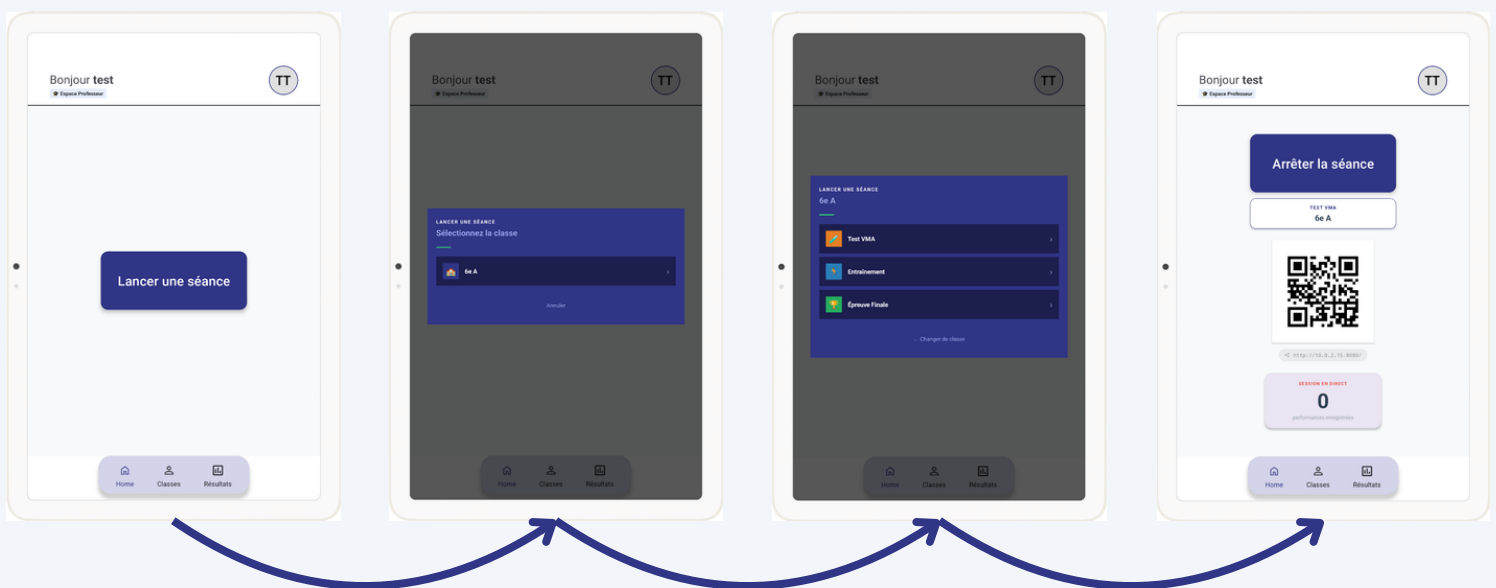
- l'application Android côté enseignant,
- le serveur local embarqué (Ktor),
- l'interface web accessible aux élèves,
- le réseau local utilisé pendant les séances.

Les tests ont été réalisés dans des conditions proches de l'utilisation réelle, sur tablette et smartphones Android, sans connexion Internet.

TESTS D'INTÉGRATION

Test 1 – Lancement d'une séance et génération du QR code

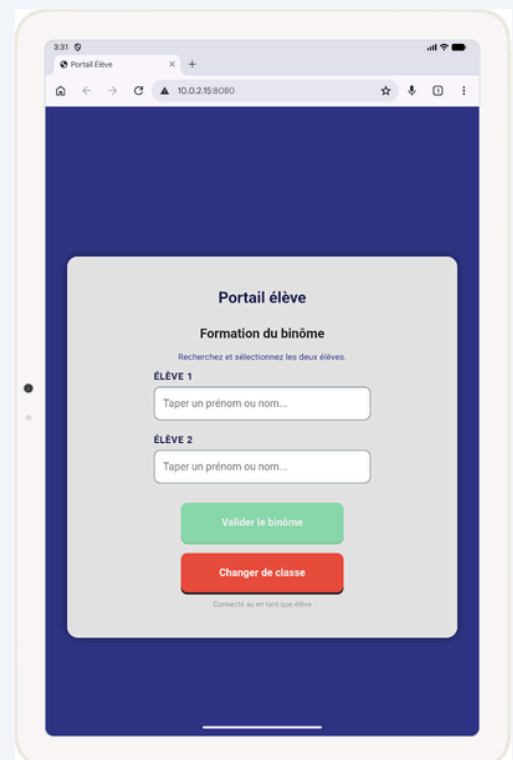
- Objectif : vérifier la création complète d'une séance et l'affichage du QR code
- Préconditions : enseignant connecté à l'application.
- Résultat attendu : le QR code est affiché et peut être scanné par un terminal élève.
- Résultat obtenu : conforme.
- Statut : Validé.



TESTS D'INTÉGRATION

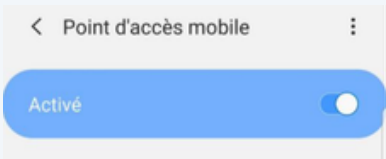
Test 2 – Accès à l'interface web élève via le QR code

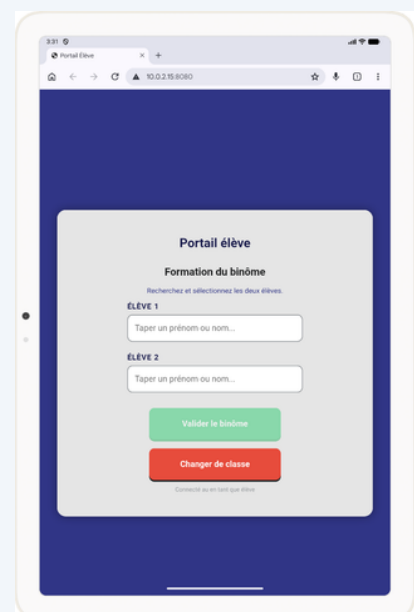
- Objectif : vérifier l'accès à l'interface web élève depuis un terminal externe.
- Préconditions : séance active et QR code affiché.
- Étapes :
 - Scanner le QR code avec un smartphone élève.
 - Vérifier l'ouverture automatique du navigateur.
- Résultat attendu : l'interface web élève s'affiche correctement.
- Résultat obtenu : conforme.
- Statut : Validé.



TESTS D'INTÉGRATION

Test 3 – Fonctionnement via le réseau local sans connexion Internet

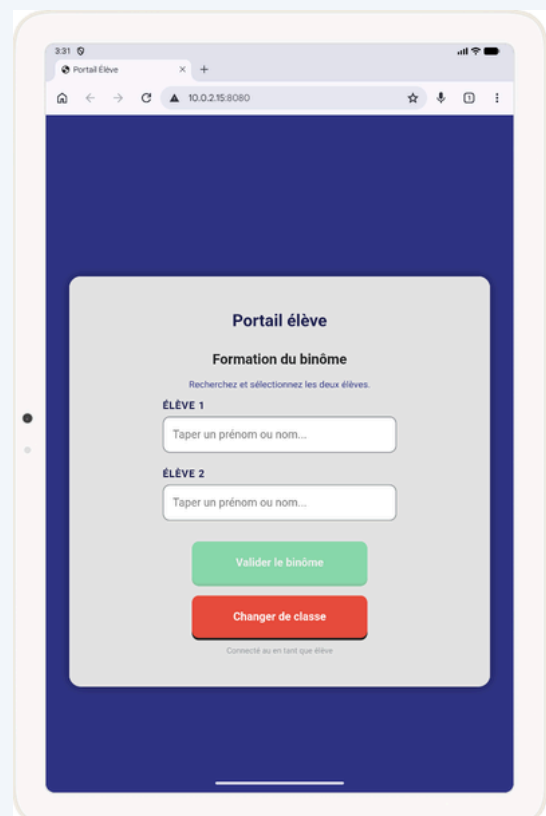
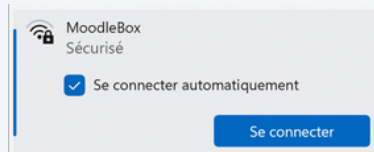
- Objectif : vérifier le fonctionnement de l'application en environnement hors ligne.
- Préconditions : hotspot Wi-Fi activé sur la tablette enseignant.
- Étapes :
 - Connecter le smartphone élève au Wi-Fi de la tablette enseignant.
 - Désactiver les données mobiles sur le téléphone élève.
 - Scanner le QR code et accéder à l'interface web.
- Résultat attendu : l'accès à l'interface élève fonctionne uniquement via le réseau local.
- Résultat obtenu : conforme.
- Statut : Validé.



TESTS D'INTÉGRATION

Test 4 – Fonctionnement via le raspberry

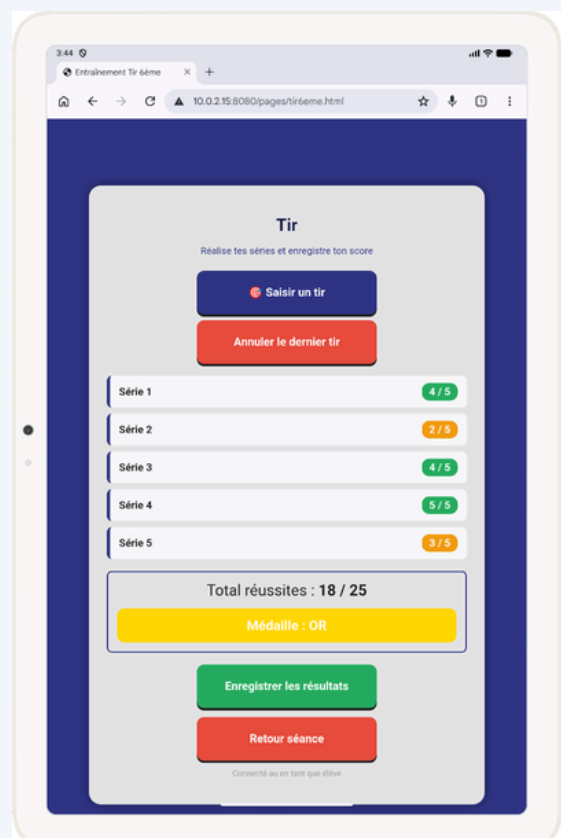
- Objectif : vérifier le fonctionnement de l'application en étant connecté au wifi du raspberry
- Préconditions : le raspberry est sur secteur et émet le wifi "moodlebox"
- Étapes :
 - Connecter la tablette élève au wifi "moodlebox"
 - Scanner le QR code et accéder à l'interface web.
- Résultat attendu : l'accès à l'interface élève fonctionne.
- Résultat obtenu : conforme.
- Statut : Validé.



TESTS D'INTÉGRATION

Test 5 – Saisie des résultats côté élève

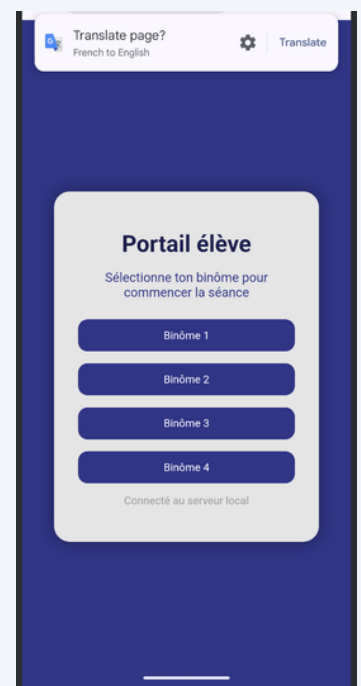
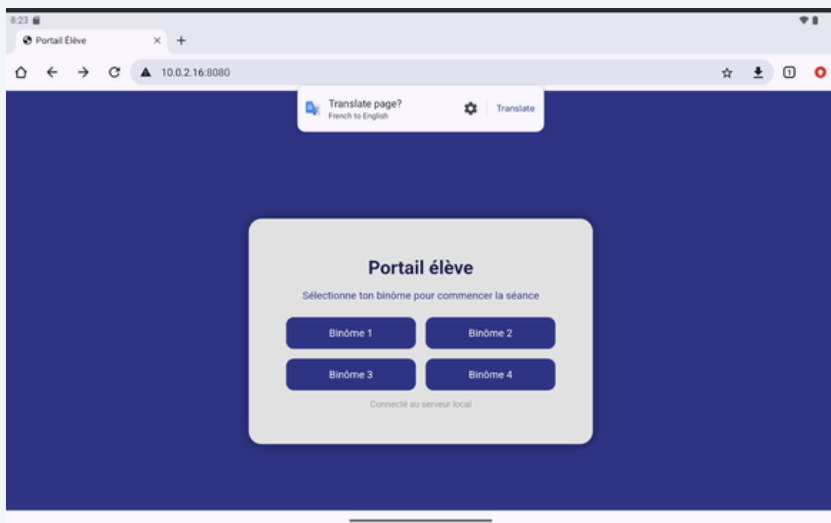
- Objectif : vérifier la prise en compte des résultats saisis par l'élève via l'interface web.
- Préconditions : interface web élève accessible.
- Étapes :
 - Renseigner le nombre de tirs réussis.
 - Valider la saisie.
- Résultat attendu : les résultats sont traités automatiquement et affichés à l'écran.
- Résultat obtenu : conforme.
- Statut : Validé.



TESTS D'INTÉGRATION

Test 6 – Connexion simultanée de plusieurs élèves

- Objectif : vérifier la stabilité du système avec plusieurs élèves connectés en même temps.
- Préconditions : séance active.
- Étapes :
 - Plusieurs élèves scannent le QR code.
 - Chaque élève accède à l'interface web et saisit ses résultats.
- Résultat attendu : aucune erreur ou déconnexion observée, les interfaces fonctionnent simultanément.
- Résultat obtenu : conforme.
- Statut : Validé.



TESTS DE SECURITE

Objectif des tests de sécurité

L'objectif est de garantir la confidentialité des données personnelles des élèves (conformité RGPD), d'assurer la disponibilité du service en milieu scolaire (souvent instable techniquement) et de maintenir l'intégrité des résultats sportifs en empêchant toute modification accidentelle ou malveillante.

Sécurité Réseau & Confidentialité (RGPD)

- **Isolation** : L'application fonctionne sur un réseau local privé (Wi-Fi émit par le Raspberry). L'absence de connexion à Internet élimine les risques d'attaques externes.
- **Contrôle d'Accès (CORS)** : Le serveur Ktor est configuré pour n'accepter que les requêtes provenant de l'interface officielle de l'application, bloquant ainsi les tentatives d'injection via des scripts tiers.
- **Flux Unidirectionnel (SSE)** : Le monitoring du professeur utilise un flux sortant exclusif. Les élèves peuvent envoyer leurs scores, mais il leur est techniquement impossible de lire ou modifier les résultats des autres binômes.

TESTS DE SECURITE

Contrôle des Saisies (Robustesse)

- **Validation par Regex (voir capture d'écran) :** Utilisation d'expressions régulières strictes pour les champs de l'application, empêchant l'injection de caractères spéciaux ou de scripts malveillants.
- **Fiabilité des Saisies Utilisateurs (voir capture d'écran) :** Filtrage automatique des scores (ex: blocage strict entre 0 et 5 pour le tir) afin de garantir la cohérence des barèmes.
- **Gestion de Complétude :** Algorithme bloquant l'envoi JSON si tous les champs requis ne sont pas remplis, évitant ainsi les erreurs de calcul (VMA/Moyenne).

Continuité de Service (Fiabilité)

- **Persistance localStorage (voir capture d'écran) :** Sauvegarde en temps réel de la progression dans le navigateur. En cas de rafraîchissement accidentel ou de coupure Wi-Fi, les données sont récupérées instantanément.
- **Gestion des Erreurs (Feedback) :** Alertes visuelles dynamiques (champs en rouge, messages d'erreur) pour guider l'utilisateur et assurer la qualité des données collectées.

TESTS DE SECURITE

Captures d'écran

Validation par Regex

Fiabilité des Saisies Utilisateurs

The screenshot shows a registration form with the following fields: 'Nom:', 'Prenom:', 'test@gmail.com', and two password fields represented by asterisks. A blue button labeled 'S'inscris' is at the bottom. Below the button is a link: 'Déjà un compte ? Se connecter'. At the very bottom, a red circle highlights a small icon and the text 'Tous les champs sont obligatoires'.

The screenshot shows a shooting game interface. At the top, it says 'Tir' and 'Réalise tes séries et enregistre ton score'. There are buttons for 'Saisir un tir' and 'Annuler le dernier tir'. A modal window titled 'Tir Série n°5' is open, asking 'Combien de cibles touchées ? (0 à 5)' with a row of buttons from 0 to 5. Below the modal, it shows 'Total réussites : 13 / 20' and 'Médaille : ARGENT'. At the bottom, there are buttons for 'Enregistrer les résultats' and 'Retour séance'.

Persistance localStorage

The screenshot shows a browser's developer console with the 'Stockage' (Storage) tab selected. It displays a table of data stored in localStorage for the URL 'http://localhost:63343'.

Key	Value
binome_actif	Binôme 1
binome_id	1
tir_medaille	OR
tir_total	21



MERCI



PROPOSÉE À : Thierry LEGOFF

[Lien Notion](#)

- 3 AVRIL 2026 -

Proposée par :

Marin Weis
Nolann Lescop
Matthieu Gouélo
Lucien Carré